KINGDOM OF SAUDI ARABIA
Ministry of Higher Education
Taibah University
College of Computer Science and Engineering (Girls Section)

جامعة طيبة
TAIBAH UNIVERSITY

المملكة العربية السعودية
وزارة التعليم العالي
جامعة طيبة
كلية علوم وهندسة الحاسب الآلي

AI331 – Machine Learning Group Project

Section: F4

ID	Student name	Student role
4455224	ABRAR ALMUTIRI	Supervised algorithm + visualization + accuracy
4456347	BUSHRA ALMUTRAFI	Supervised algorithm + visualization + accuracy
4455088	RANA ALENEZI	Supervised algorithm + Data Cleaning
4456055	SHATHA GHABBAN	Unsupervised algorithm (K-means)
4454973	NUHA ALMUQIT	Supervised algorithm + Data Cleaning

Supervised by D. Raghdah

November 2025

KINGDOM OF SAUDI ARABIA
Ministry of Higher Education
Taibah University
College of Computer Science and Engineering (Girls Section)

جامعة طيبة
TAIBAH UNIVERSITY

المملكة العربية السعودية
وزارة التعليم العالي
جامعة طيبة
كلية علوم وهندسة الحاسب الآلي

Table of Contents

1. Abstract and Introduction 3
2. Dataset and Domain Description 4
3. Theoretical Background 5
   3.1 Supervised models 5
   3.2 Unsupervised model 6
4. Methodology and challenges 7
   4.1 Data Preprocessing Pipeline 8
   4.2 Supervised models(Linear regression) 10
   4.3 Supervised models(Classification) 14
   4.4 Unsupervised model 16
5. Results and Evaluation 23
6. Extra 33
   K-Nearest Neighbors (KNN) – Classification Model 34
   Gradient Boosting Regressor(sleptim1) 36
   Random Forest 37
   XGBoost 39
   Comparision of Classification Models 41
   Hierarchical Model 42
   Clustering with DBSCAN: 44
7. Discussion and Conclusion 45
8. References 47

1. Abstract and Introduction

- This project involved leveraging a large dataset to analyze features and extract patterns. We deployed multiple models to generate significant predictions, clustering, and classifications based on the database.

- The domain is healthcare, specifically behavioral health surveillance, which focuses on studying the impact of lifestyle and daily habits on long-term physical health. The dataset is based on the Behavioral Risk Factor Surveillance System (BRFSS). Machine learning plays a critical role in this field by enabling the analysis of large and complex health datasets that traditional statistical methods struggle to handle.

- We are working on develop and evaluate machine learning models to predict and classify individual lifestyles into healthy or unhealthy lifestyle based on behavioral factors. identifying the most influencing features (such as diet , sleep hours ,etc.) to classify the lifestyle. The scope: -Including data preprocessing, feature selection, model training, and evaluation. -Two supervised learning algorithms(regression and decision tree) and One unsupervised learning algorithm(K-means) , both algorithms Will be implemented to analyze and compare classifications performance.

2. Dataset and Domain Description

•The Dataset is **Behavioral Risk Factor Surveillance System (BRFSS)** [1] 2013 in Kaggle **.** It focused on adult aged and above from the United States in 2013 and excludes incomplete records, the findings aim to provide insights into lifestyle risk patterns ,it contains approximately 500,000 and around 300 features. Most of the variables are numerical and other variables are categorical, usually represented by codes or binary values and a few columns are stored as object or string types, which contain non-numeric information.

•BRFSS is an ongoing surveillance system designed to measure behavioral risk factors for the non-institutionalized adult population (18 years of age and older) residing in the US. The BRFSS was initiated in 1984, with 15 states collecting surveillance data on risk behaviors through monthly telephone interviews. From this dataset, several types of insights can be extracted using different machine learning approaches In Regression , classification and clustering. Such as predicting the risk of diabetes and quality of life.

• There is a high possibility of underfitting due to the noise and irrelevant of the data and an imbalance of data , which may also lead to the curse of dimensionality. Since the dataset contains many missing values, this could cause problems during training or testing. We will work on these issues using several methods, such as feature extraction (**PCA**), and feature selection (**selectBestK**)filling in the missing values, and applying the concept of cross-validation.

3. Theoretical Background

3.1 Supervised models

The **Linear Regression** model was chosen in this analysis because it is one of the most effective and easily interpretable methods for modeling relationships between continuous numerical variables. After applying PCA, the data were transformed into a set of continuous, orthogonal numerical components free from high correlations, which are the ideal conditions for Linear Regression to perform efficiently.

$$\hat{Y} = bX + a$$

The models were built using the principal components extracted from PCA as input features, as they capture the majority of the variance in the original dataset while removing redundant or highly correlated information. This contributes to a more stable regression process and ensures that the model learns genuine linear patterns rather than noise.

Each model was trained to predict a specific target variable (weight2, sleptim1, or X_vegesum) based on the linear relationship between that variable and the principal components. The decision to use Linear Regression was also supported by the nature and size of the dataset, where the large number of observations combined with a set of high-quality features allows the model to generalize effectively and produce reliable predictions. Linear Regression was selected because it aligns well with the data after transformation via PCA, provides clear interpretability, and leverages the linear relationships revealed by PCA to build robust predictive models.

We applied a **Decision Tree** (classification model) to classify Diabetes, a chronic condition affecting blood sugar regulation, three into categories: non-diabetic, at risk (pre-diabetes,

borderline, pregnancy), and diabetic. The Decision Tree model is well-suited for this problem due to its interpretability, which allows easy understanding of the decision-making process, and its ability to handle non-linear relationships between variables like age and BMI.

The Gini Index is used in decision trees to measure the “impurity” of node in the tree:

For K classes, the Gini index at node m is:

$$I_m^{Gini} = 1 - \sum_{i=1}^{K} (p_m^i)^2$$

It indicates how often a randomly selected element would be misclassified if labeled according to the distribution of labels in that node.

In a Decision Tree model, the decision boundaries are orthogonal, meaning split the data based on specific feature values. This makes the model simple and easy to interpret.

Decision Trees are non-parametric models, meaning they do not depend any assumptions about the distribution of the data.

3.2 Unsupervised model

Pattern Recognition and Relationship Discovery: We utilized K-Means $d(x+y) = \sqrt{\sum_{i=1}^{n}(x-y)}$ to identify intrinsic structures within the data that are not immediately apparent, allowing us to map out complex relationships between variables.

Optimization by using Elbow Method:

To avoid arbitrary selection of clusters, we applied the Elbow Method. This involved plotting the variance against the number of clusters and identifying the "elbow" point, ensuring the model achieves the best balance between cohesion and separation.

Behavioral Classification: Individuals were categorized into clusters based on key feature sets. This allowed us to profile each group according to their specific behaviors and traits, differentiating "high-value" or "active" users from others.

4. Methodology and challenges

4.1 Data Preprocessing Pipeline

At the beginning, we loaded the dataset and imported all the required libraries.
The preprocessing pipeline started by exploring the data to understand the different types of variables and their distributions before performing cleaning and transformations.

The pipeline included the following core steps:
dropping irrelevant columns – handling missing values – handling outliers/scaling – encoding – feature selection – PCA.

The dataset contains many attributes, but several of them were unnecessary for the machine learning models (such as statistical information about the sampling process, insurance-related fields, and financial data). These columns were grouped into a list and removed because they do not contribute to predicting the target variable.

For numerical columns that had similar meaning or high similarity, we used the Correlation Coefficient to detect and remove highly correlated features. After that, a duplicate-removal function was applied to ensure the dataset contained only unique records.

When handling missing values, there were two options:
filling or dropping the missing rows.
For our case, filling was the better choice. Since the dataset contains both numerical and categorical (object) data:

Numerical columns: we used the median(), because it is more robust to outliers.

Categorical columns: we used the mode(), which is the most frequent value.

A critical step before applying any machine learning model is ensuring that all data is numerical, because models rely on mathematical computation.
We used four types of encoding, depending on the nature of the column:

Label Encoding: for unordered categorical values (e.g., Male/Female, Yes/No).

Ordinal Encoding: for categorical values that naturally follow an order.

One-Hot Encoding: for categorical values where order does not matter and should not be encoded numerically.

Map Encoding: for important columns with long or complex categories that required custom mapping.

After converting all categorical data into numerical form, we prepared the dataset for modeling by reducing its dimensionality.

At the first We applied Feature Selection using **SelectKBest**.
This method evaluates all features (except the target) and selects the top k most useful ones.
To validate the selected features, we trained a Linear Regression model to measure accuracy.
The results showed low accuracy, which indicated that Feature Selection alone was not sufficient.

Therefore, We used Principal Component Analysis (**PCA**) to reduce the number of features from 156 to a smaller set of principal components. The goal was to simplify the data while preserving the most important variances. This simplification makes the model more stable, easier to train and interpret, and reduces computational complexity without losing critical information.
After applying PCA, we trained the model again, and this time the accuracy significantly improved showing that PCA was much more effective for this dataset.

4.2 Supervised models(Linear regression)

Linear Regression is one of the simplest and most widely used models for capturing linear relationships between variables. To implement this model, we first separated the dataset into features (X) and targets (Y), then split them into training and testing subsets to ensure proper evaluation. After importing the Linear Regression library, the model was trained on the training data (X_train) and later used to generate predictions on unseen test data.

```
#linear
ysl = df_reduced['sleptim1']

X_trainsl, X_testsl, y_trainsl, y_testsl = train_test_split(
    X_pca, ysl, test_size=0.2,random_state=42)

reg_modelsl = LinearRegression()
reg_modelsl.fit(X_train, y_train)
y_predsl = reg_modelsl.predict(X_test)
```

To measure model performance, we used the **R^2 score**, which compares the predicted values with the actual ground-truth values. We applied Linear Regression to three different target variables — **sleptim1, weight**, and **X_vegesum** — and evaluated the accuracy for each prediction task.

```
#linear
ysl = df_reduced['sleptim1']

X_trainsl, X_testsl, y_trainsl, y_testsl = train_test_split(
    X_pca, ysl, test_size=0.2,random_state=42)

reg_modelsl = LinearRegression()
reg_modelsl.fit(X_train, y_train)
y_predsl = reg_modelsl.predict(X_test)

print("R2 score for Generalizing :", r2_score(y_testsl,y_predsl))
print('\nMean Squared Error of linear regreesion model(sleep time):',mean_squared_error(y_testsl, y_predsl) )
print('Mean Absolute Error of linear regreesion model(sleep time):',mean_absolute_error(y_testsl, y_predsl))
✓ 0.7s                                                                    Python
R2 score for Generalizing : -0.04495851898306635

Mean Squared Error of linear regreesion model(sleep time): 1.650289154922591 8
Mean Absolute Error of linear regreesion model(sleep time): 0.995477229310734 8
```

```
yw= df_reduced['weight2']

Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    X_pca3, yw, test_size=0.2, random_state=42
)

modelw = LinearRegression()
modelw.fit(Xw_train, yw_train)

y_predw = modelw.predict(Xw_test)

#counting score of training phase
y_pred_trwe=modelw.predict(Xw_train)
r2_train_we = r2_score(yw_train, y_pred_trwe)


print("R2 score for training :", r2_train_we)
print("R2 score for Generalizing:", r2_score(yw_test, y_predw))
print('Mean Squared Error of linear regreesion model(sleep):',mean_squared_error(yw_test, y_predw) )
print('Mean Absolute Error of linear regreesion model(sleep):',mean_absolute_error(yw_test, y_predw))
✓ 2.2s                                                                    Python
R2 score for training : 0.7123581242957993
R2 score for Generalizing: 0.7135526245766299
Mean Squared Error of linear regreesion model(sleep): 404.2928988933964 5
Mean Absolute Error of linear regreesion model(sleep): 14.48987622589616 8
```

```python
yv = df_reduced['X_vegesum']

Xv_train, Xv_test, yv_train, yv_test = train_test_split(
    X_pca4, yv, test_size=0.2, random_state=42
)

modelv = LinearRegression()
modelv.fit(Xv_train, yv_train)

y_predv = modelv.predict(Xv_test)

#counting score of training phase
y_pred_trv=modelv.predict(Xv_train)
r2_train_veg = r2_score(yv_train, y_pred_trv)


print("R2 score for training :", r2_train_veg)
print("R2 score for Generlizing:", r2_score(yv_test, y_predv))
print('Mean Squared Error of linear regreesion model(X_vegesum):',mean_squared_error(yv_test, y_predv) )
print('Mean Absolute Error of linear regreesion model(X_vegesum):',mean_absolute_error(yv_test, y_predv))
```

✓ 2.3s Python

```
R2 score for training : 0.6346771227179059
R2 score for Generlizing: 0.6345338893025716
Mean Squared Error of linear regreesion model(X_vegesum): 2779.7299350420963
Mean Absolute Error of linear regreesion model(X_vegesum): 34.023299940228
```

To further improve reliability, we applied K-Fold [2] Cross-Validation, where the dataset is divided into K equal folds. The model is trained and tested K times, each time using a different fold as the test set. This provides a more stable estimate of performance compared to a single train-test split.

with new goal "weight", implement cross validation to improve performance

```python
kf = KFold(n_splits=10, shuffle=True, random_state=42)

scores = cross_val_score(modelw,X_pca3, yw, cv=kf, scoring='r2')

print(f"R2 for each fold: {scores}")

average_accuracy = np.mean(scores)
print(f"Average R2: {average_accuracy:.2f}")
```

✓ 21.8s

```
R2 for each fold: [0.71455996 0.71251549 0.71495898 0.7124615  0.7105867  0.7105749
 0.71007944 0.71327778 0.71087832 0.71469717]
Average R2: 0.71
```

using k-Fold would help to improve the performance

```python
kf = KFold(n_splits=10, shuffle=True, random_state=42)


scores = cross_val_score(reg_modelsl,X_pca, ysl, cv=kf, scoring='r2')

print(f"R2 for each fold: {scores}")

average_accuracy = np.mean(scores)
print(f"Average R2: {average_accuracy:.2f}")
```

✓ 17.2s

```
R2 for each fold: [0.05002412 0.05384926 0.05629008 0.05380857 0.05601248 0.05247532
 0.0520132  0.04940045 0.04943925 0.05429469]
Average R2: 0.05
```

with new goal "X_vegesum", implement cross validation to improve performance

```python
kf = KFold(n_splits=10, shuffle=True, random_state=42)

scores = cross_val_score(modelv,X_pca4, yv, cv=kf, scoring='r2')

print(f"R2 for each fold: {scores}")

average_accuracy = np.mean(scores)
print(f"Average R2: {average_accuracy:.2f}")
```

✓ 21.2s

```
R2 for each fold: [0.63453638 0.63452592 0.63428462 0.63672977 0.63501232 0.63195689
 0.63412292 0.63431731 0.63607487 0.63349163]
Average R2: 0.63
```

R^2 score

$$\text{R-Squared} = 1-\left(\frac{SSR}{SST}\right)=1-\frac{\sum_{i=1}^{n}(yi-\widehat{yi})^2}{\sum_{i=1}^{n}(yi-\overline{yi})^2}$$

Mean Squared Error (MSE)

Mean Absolute Error (MAE)

Challenges:
The dataset used in this project is highly noisy and shows weak or unclear relationships among variables. Correlation analysis and scatter plots revealed that many features do not meaningfully predict the target values. This issue was especially severe for sleptim1, where almost no machine learning model was able to achieve a reasonable accuracy.

The poor performance reflects the inherent randomness and low predictive signal in the data itself, rather than a limitation of the algorithms.

Next, in the extra “Gradient Boosting” experiment, we applied the model specifically to the sleptim1 goal, yet the results still showed no improvement. Regardless of the algorithm used, sleep duration produced consistently poor accuracy, suggesting that the dataset simply does not contain a strong predictive signal for this variable.

4.3 Supervised models(Classification)

DECISION TREE

```
#PCA for Classification
X_1= df_reduced.copy()
targets1 = ['diabete3_encoded']
X_1 = df_reduced.drop(columns=targets, errors='ignore')

scaler1 = StandardScaler()
X1_scaled = scaler.fit_transform(X_1)


pca1 = PCA(n_components=0.95)
X_pca1 = pca1.fit_transform(X1_scaled)
```

Principal Component Analysis (PCA):

was applied to reduce the dimensionality of the dataset while retaining 95% of the variance. First, the target variable was separated from the features. Then, **StandardScaler** was used to scale the features, ensuring that each feature has a mean of 0 and a standard deviation of 1. Finally, PCA was applied to transform the data, reducing the number of features while preserving essential information for classification.

```
#Decision Tree
y_classi = df_reduced['diabete3_encoded']
xtr, xte, ytr, yte = train_test_split(X_pca1, y_classi, random_state = 42, stratify = y_classi)

dt = DecisionTreeClassifier(max_depth=7, random_state = 42)
```

train_test_split():

X_pca1: The features, which have been transformed using PCA.

y_**classi:** The target variable

random_state=42: Ensures reproducibility by fixing the random seed.

stratify=y_classi: Ensures that the class distribution is maintained in both training and testing sets, which is important for imbalanced data.

DecisionTreeClassifer():

max_depth = 7

We selected a maximum depth of 7 to balance the trade-off between model complexity and generalization.
A deeper tree tends to overfit, while a shallow tree may underfit. After experimenting with several depths, a depth of 7 provided a good compromise and gives good accuracy.

random_state = 42

A fixed random seed (42) was used to ensure reproducibility.

Using the same random state allows the train/test split and model results to stay consistent

```
import time

start_time_dt = time.time()
dt.fit(xtr, ytr)
end_time_dt = time.time()
dt_train_time = end_time_dt - start_time_dt
print('training time:',dt_train_time)

y_pred_dt = dt.predict(xte)

training time: 6.08510422706604
```

measures the training time for the Decision Tree model:

Record the start time using time.time(),Train the model using dt.fit(xtr, ytr) then Record the end time after training. Then Calculate the training time then print it which in this case is 6.085 seconds.

y_pred_dt = dt.predict(xte) :

makes predictions using the trained Decision Tree model on the test data (xte) and stores the results in y_pred_dt.

4.4 Unsupervised model

Data Preprocessing and Dimensionality Reduction

To ensure data integrity, a copy of the original dataset was created, and column names were preserved for future reference.

Subsequently, feature scaling was applied using the StandardScaler to normalize the data distribution; this step is crucial to prevent features with larger magnitudes from biasing the distance calculations.

Initially, Principal Component Analysis (PCA) was applied to reduce dimensionality. However, the method yielded a suboptimal performance score of 0.39. This low score indicates that the dataset contains complex, non-linear relationships that a linear technique like PCA cannot effectively capture. Consequently, we shifted to UMAP (Uniform Manifold Approximation and Projection).

```
scaler = StandardScaler()
dfS = scaler.fit_transform(copy_of_orignal)


APCS = PCA(n_components=2)
DF_PCA = APCS.fit_transform(dfS)
```

```
#we calucate the score by using silhouette_score
#we choose the best clusters based on the maximum accuracy

score = silhouette_score(DF_PCA, kmeansModel.labels_)
score= round(score,2)
score
```

```
0.39
```

Unlike PCA, UMAP [3]is a manifold learning technique designed to preserve both local and global structures in non-linear data, making it far more suitable for this specific dataset.

Uniform Manifold Approximation and Projection (UMAP) algorithm was utilized with a fixed random seed (random_state=42) to ensure reproducibility.

This reduced the high-dimensional data into a 2-dimensional space, resulting in a final output shape of (491775, 2), which facilitates effective visualization and analysis of the underlying data structure.

```
#We applied feature scaling to avoid misinterpreting feature importance
scaler = StandardScaler()
features_scaled = scaler.fit_transform(copy)
```

```
#We applied UMAP to focus on studying the relationships within the data
#We utilized UMAP for dimensionality reduction and visualization
UM1 = umap.UMAP(random_state=42)
umapModel = UM1.fit_transform(features_scaled)
```

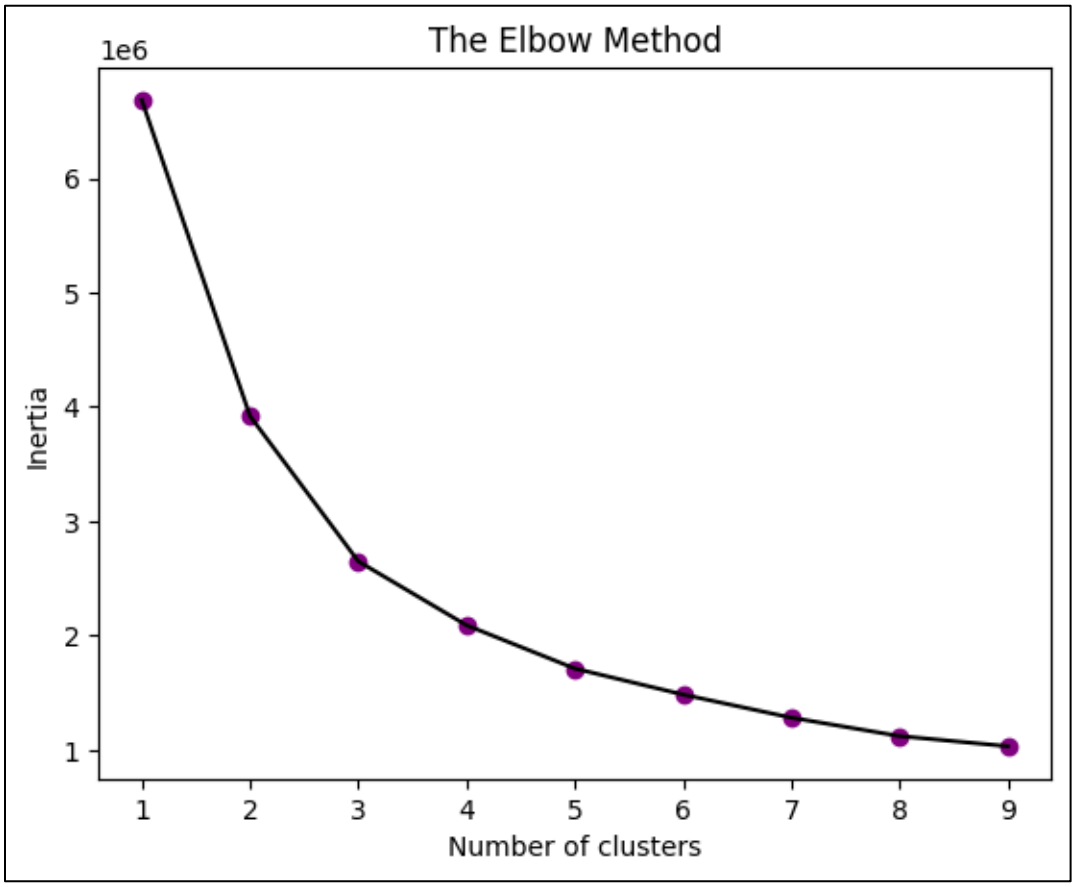
```
umapModel.shape
```

```
(491775, 2)
```

```
score = silhouette_score(umapModel, kmeansModel.labels_)
score= round(score,2)
score
```

```
np.float32(0.52)
```

To determine the optimal number of clusters (**K**) we utilized the Elbow method, which indicated a potential range of 2 to 4 clusters. Further evaluation revealed that **K=2** yielded the best clustering structure and performance metrics.



Upon analyzing the average values of features of each group, we observed a distinct pattern: Following this**,** we extracted the top 10 distinctive features per cluster for detailed analysis.

Based on this feature-level examination, we were able to interpret the characteristics of each group and determine the appropriate category or name for each cluster

```
Top 10 features for cluster 0:
hivtstd3    92009.000000
wtkg3        7998.101238
pafreq1_     3177.362693
maxvo2_      2919.917522
pafreq2_     2000.000000
height3       505.659182
strfreq_      489.521486
longwtch      406.000000
fvbeans       238.804103
fvorang       238.762325
Name: 0, dtype: float64
```

```
Top 10 features for cluster 1:
hivtstd3    92009.000000
wtkg3        7899.993431
pafreq1_     3147.671195
maxvo2_      2802.398913
pafreq2_     2000.000000
height3       505.629239
strfreq_      428.871059
longwtch      406.000000
fvbeans       241.364652
fvorang       241.035445
Name: 1, dtype: float64
```

```
Top 10 features for cluster 2:
hivtstd3    92009.000000
wtkg3        7786.142894
pafreq1_     3263.817985
maxvo2_      2872.000264
pafreq2_     2000.000000
strfreq_      623.283823
height3       505.864975
longwtch      406.000000
fvbeans       239.739278
fvorang       236.876777
Name: 2, dtype: float64
```

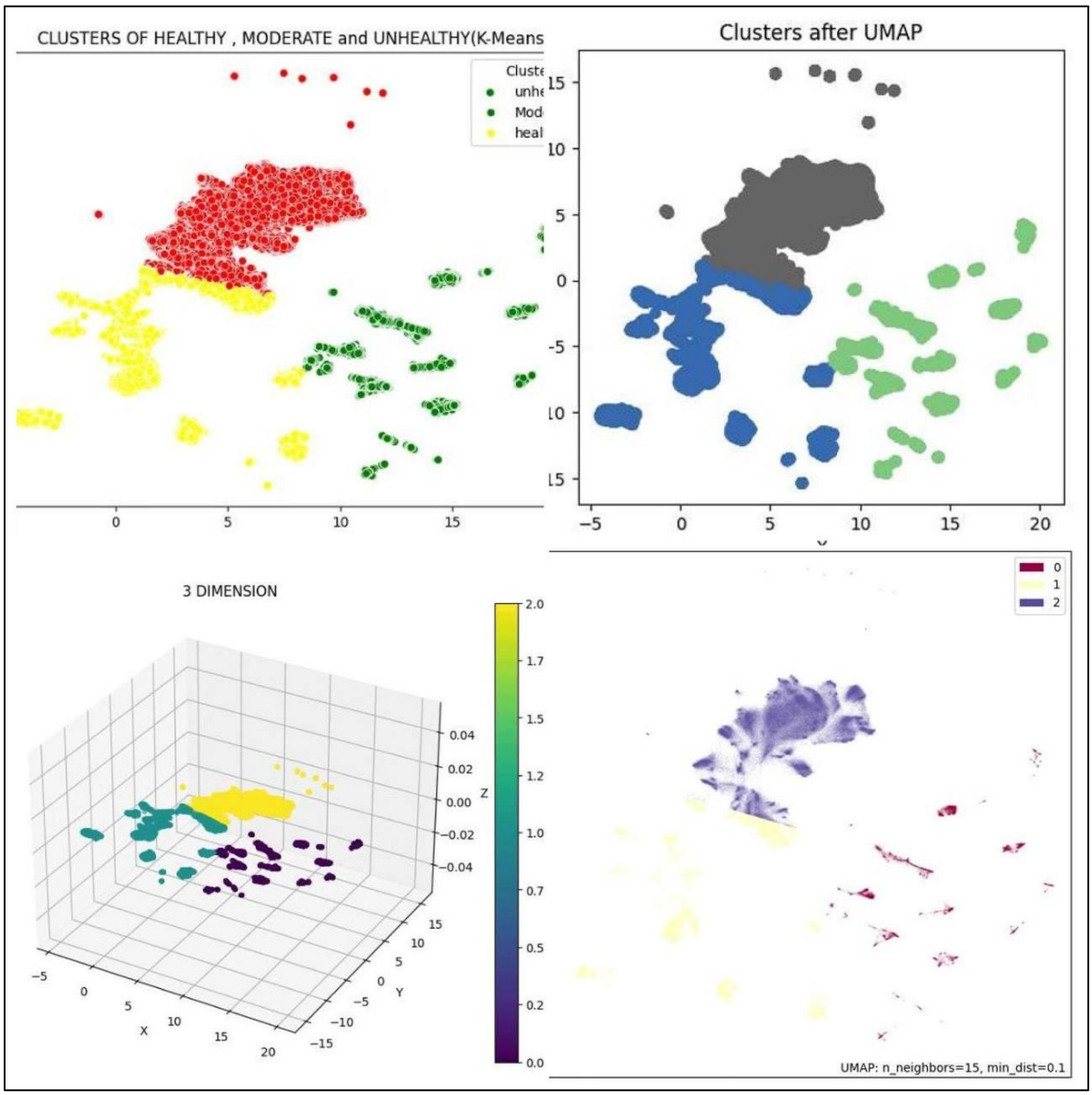
Analysis of the means per cluster confirms our segmentation:

Cluster 0 (The Healthy Group): This cluster represents the most physically fit segment of the population. Individuals in this group are characterized by the lowest average body weight and the highest level of engagement in physical activities, particularly strength training. Their overall profile suggests a lifestyle consistently dedicated to fitness, regular exercise, and effective weight management.

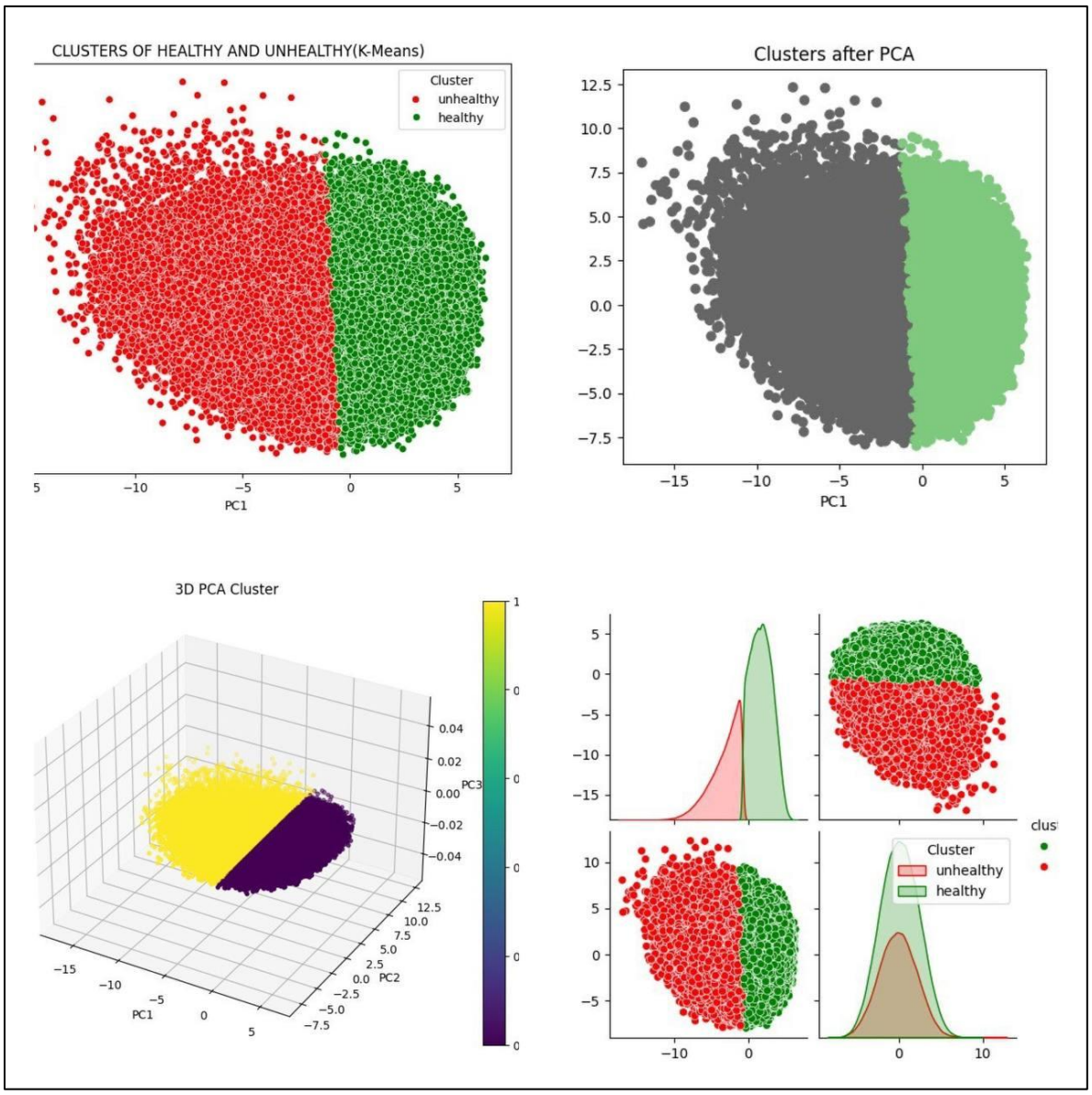
Cluster 1 (The Moderate Group): This group serves as the intermediate profile, bridging the gap between the fit and the at-risk segments. Their physical metrics, including weight and exercise frequency, fall within a middle range. This indicates a lifestyle that maintains a baseline level of activity but lacks the comprehensive and intensive fitness regimen observed in the healthy group.

Cluster 2 (The Unhealthy Group): This segment exhibits the least favorable health indicators among the three groups. Members of this cluster are characterized by metrics associated with higher health risks, such as increased body weight and significantly lower engagement in physical activities compared to the healthy group. This profile strongly points towards a more sedentary lifestyle that may require intervention to improve overall well-being.

With UMAP:


CLUSTERS OF HEALTHY , MODERATE and UNHEALTHY(K-Means
Clusters after UMAP
3 DIMENSION
UMAP: n_neighbors=15, min_dist=0.1

With PCA:


CLUSTERS OF HEALTHY AND UNHEALTHY(K-Means)
Cluster
unhealthy
healthy
PC1
Clusters after PCA
PC1
3D PCA Cluster
PC1
PC2
PC3
Cluster
unhealthy
healthy

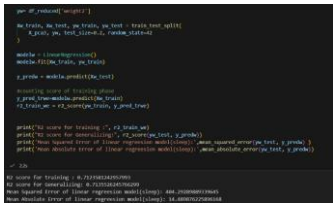
Comparison between PCA and UMAP:

- **Linearity:** PCA assumes linear relationships among features, which resulted in significant information loss (score: 0.39). In contrast, UMAP is capable of modeling complex, non-linear manifolds.
- **Structure Preservation:** While PCA focuses on preserving the global variance, it failed to represent the data's underlying structure effectively. UMAP was selected to better preserve the local neighborhoods and cluster separability within the high-dimensional space

5. Results and Evaluation

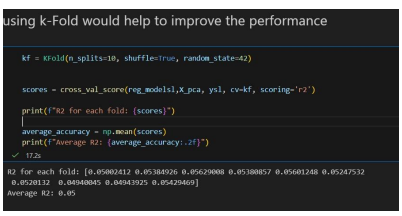
Linear regression

```
# linear
ysl = df_reduced['sleptim1']

X_trainsl, X_testsl, y_trainsl, y_testsl = train_test_split(
    X_pca, ysl, test_size=0.2,random_state=42)

reg_modelsl = LinearRegression()
reg_modelsl.fit(X_train, y_train)
y_predsl = reg_modelsl.predict(X_test)

print("R2 score for Generalizing :", r2_score(y_testsl,y_predsl))
print("\nMean Squared Error of linear regression model(sleep time):",mean_squared_error(y_testsl, y_predsl) )
print("Mean Absolute Error of linear regression model(sleep time):",mean_absolute_error(y_testsl, y_predsl))
✓ 0.7s

R2 score for Generalizing : -0.004958518983066635

Mean Squared Error of linear regression model(sleep time): 1.6502891549223918
Mean Absolute Error of linear regression model(sleep time): 0.9954772293187348
```


```
y= df_reduced['weight']

Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    X_pca, yw, test_size=0.2, random_state=42
)

modelw = LinearRegression()
modelw.fit(Xw_train, yw_train)

y_predw = modelw.predict(Xw_test)

Accuracy score of training phase
y_pred_trw=modelw.predict(Xw_train)
r2_train_wr = r2_score(yw_train, y_pred_trw)


print("R2 score for training :", r2_train_wr)
print("R2 score for Generalizing:", r2_score(yw_test, y_predw))
print("Mean Squared Error of linear regression model(sleep): ",mean_squared_error(yw_test, y_predw) )
print("Mean Absolute Error of linear regression model(sleep): ",mean_absolute_error(yw_test, y_predw))
✓ 23s

R2 score for training : 0.7125583245957993
R2 score for Generalizing: 0.7125583245796209
Mean Squared Error of linear regression model(sleep): 404.2928080919845
Mean Absolute Error of linear regression model(sleep): 14.480876525896168
```

```
yv = df_reduced['X_vegesum']

Xv_train, Xv_test, yv_train, yv_test = train_test_split(
    X_pca, yv, test_size=0.2, random_state=42
)

modelv = LinearRegression()
modelv.fit(Xv_train, yv_train)

y_predv = modelv.predict(Xv_test)

Accuracy score of training phase
y_pred_trv=modelv.predict(Xv_train)
r2_train_vg = r2_score(yv_train, y_pred_trv)


print("R2 score for training :", r2_train_vg)
print("R2 score for Generalizing", r2_score(yv_test, y_predv))
print("Mean Squared Error of linear regression model(X_vegesum): ",mean_squared_error(yv_test, y_predv) )
print("Mean Absolute Error of linear regression model(X_vegesum): ",mean_absolute_error(yv_test, y_predv))
✓ 23s

R2 score for training : 0.6186775227179059
R2 score for Generalizing: 0.6186774804926759
Mean Squared Error of linear regression model(X_vegesum): 2779.7209350620961
Mean Absolute Error of linear regression model(X_vegesum): 36.023209988228
```

K-Fold for each model


```
using k-Fold would help to improve the performance

kf = KFold(n_splits=10, shuffle=True, random_state=42)


scores = cross_val_score(reg_modelsl,X_pca, ysl, cv=kf, scoring='r2')

print(f"R2 for each fold: {scores}")

average_accuracy = np.mean(scores)
print(f"Average R2: {average_accuracy:.2f}")
✓ 17.2s

R2 for each fold: [0.05002412 0.05384926 0.05629008 0.05380857 0.05601248 0.05247532
 0.0520132  0.04940045 0.04943925 0.05429469]
Average R2: 0.05
```

```
with new goal "weight", implement cross validation to improve performance

kf = KFold(n_splits=10, shuffle=True, random_state=42)

scores = cross_val_score(modelw,X_pca3, yw, cv=kf, scoring='r2')

print(f"R2 for each fold: {scores}")

average_accuracy = np.mean(scores)
print(f"Average R2: {average_accuracy:.2f}")
✓ 21.8s

R2 for each fold: [0.71455996 0.71251549 0.71495898 0.7124615  0.7105867  0.7105749
 0.71007944 0.71327778 0.71807832 0.71469717]
Average R2: 0.71
```

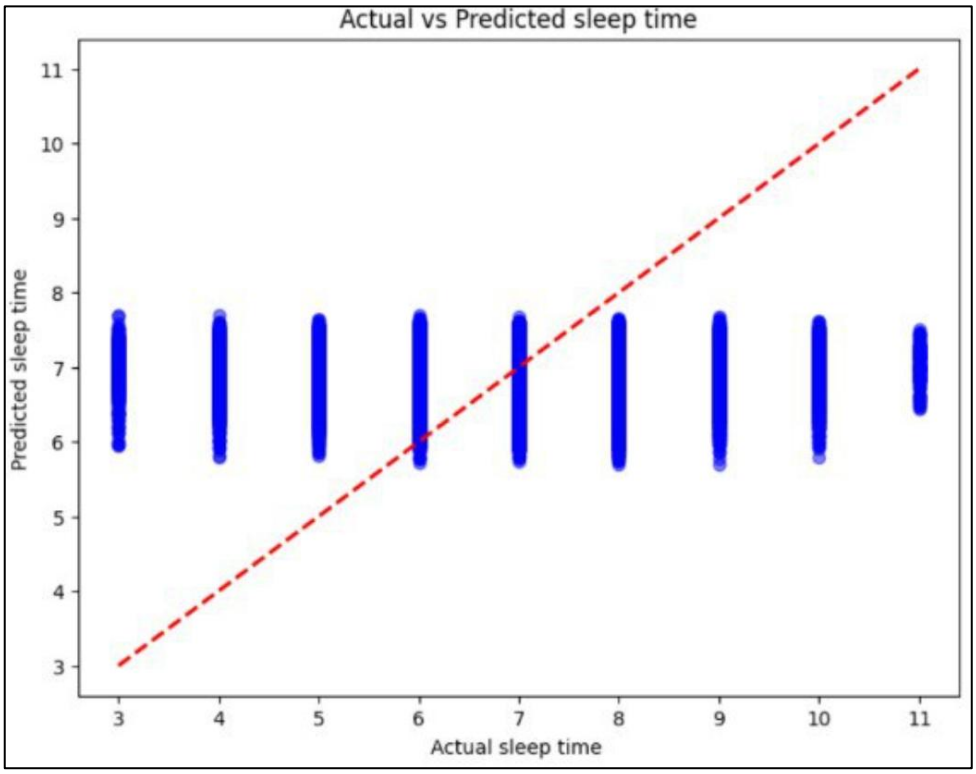
```
with new goal "X_vegesum", implement cross validation to improve performance

kf = KFold(n_splits=10, shuffle=True, random_state=42)

scores = cross_val_score(modelv,X_pca4, yv, cv=kf, scoring='r2')

print(f"R2 for each fold: {scores}")

average_accuracy = np.mean(scores)
print(f"Average R2: {average_accuracy:.2f}")
✓ 21.2s

R2 for each fold: [0.63453638 0.63452592 0.63428462 0.63672977 0.63501232 0.63195689
 0.63412292 0.63431711 0.63607487 0.63349163]
Average R2: 0.63
```

Sleep Time (sleptim1)

The Linear Regression model performed poorly on the sleep variable.

- **R^2 Score:** ≈ −0.004
- **MSE:** 1.65
- **MAE:** 0.99

These results indicate that the model is unable to generalize or capture any meaningful relationship for sleep time. The K-Fold scores also remained extremely low (Average $R^2$0.05), confirming that the dataset does not contain predictive features for this target.



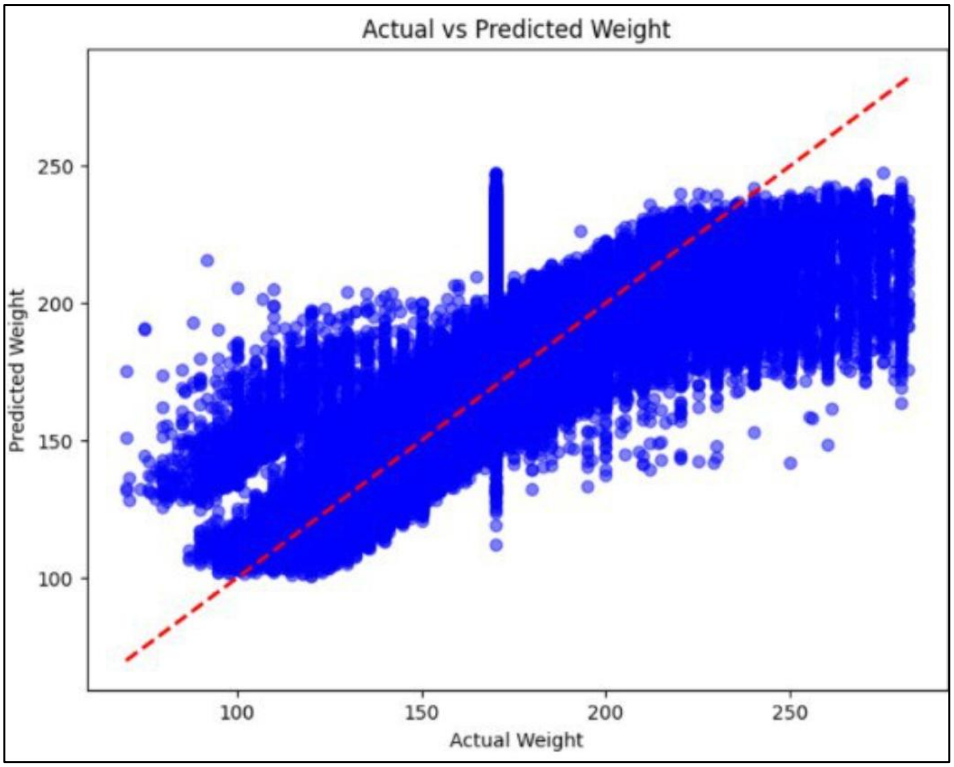
Weight Prediction

The model performed significantly better on predicting weight.

- **Training R^2:** 0.71
- **Testing R^2:** 0.71
- **MSE:** 404.23
- **MAE:** 14.48

The K-Fold Cross-Validation showed:

- **Average $R^2 \approx 0.71$**
  This demonstrates that the model generalizes well and captures a stable linear signal in the data.



Vegetable Consumption (X_vegesum)

Moderate performance:

- **Training R^2:** 0.63
- **Testing R^2:** 0.63
- **MSE:** 2779
- **MAE:** 34

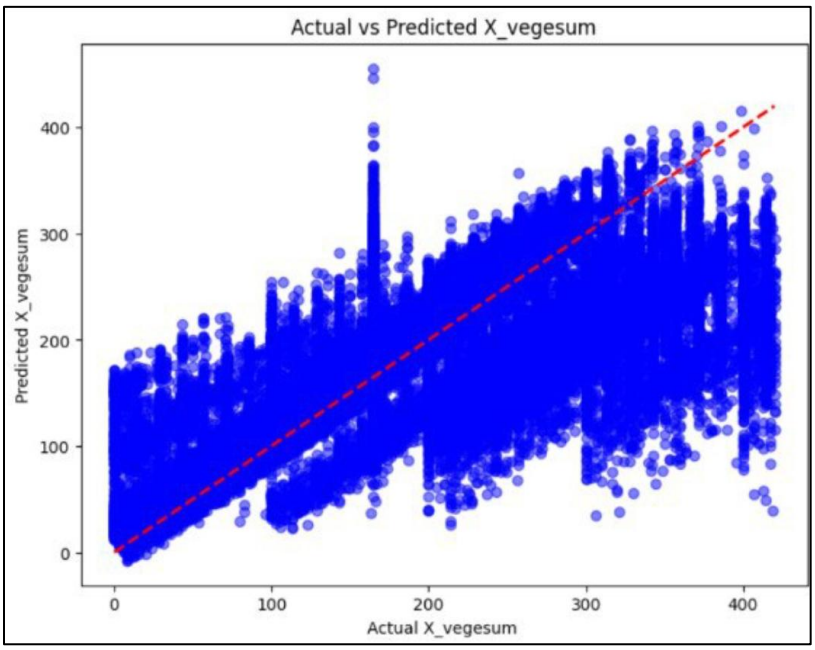
K-Fold Cross-Validation confirmed consistency:

- **Average $R^2 \approx 0.63$**

This target provided a predictable linear pattern, though weaker than weight.

For the three linear models we tested, the **weight prediction** goal gave the best results. The model reached around **R^2 = 0.71**, which means it was able to learn a good pattern from the data.

The second-best performance was for **X_vegesum**, with an **R^2 around 0.63**, showing a decent and predictable relationship.



However, the **sleep time goal (sleptim1)** performed very poorly. The Linear Regression model gave almost **R^2 = 0**, and even when we used 10-Fold Cross-Validation, the average score stayed around **R^2 = 0.05**. We also tried a stronger non-linear model, **Gradient Boosting**, but the result was still very low (**$R^2 \approx 0.04$**).

These results show that sleep time does **not have any clear relationship** with the other features in the dataset. No matter which model or validation method we used, the accuracy did not improve, meaning this target is basically unpredictable with the available data.

Overall, **weight** and **X_vegesum** were learnable goals, but **sleep time** had no meaningful pattern for any model to learn.

For decision tree:

```
# Accuracy & report
from sklearn.metrics import classification_report
print("Decision Tree Accuracy:", accuracy_score(yte, y_pred_dt))
print("\nClassification Report:\n", classification_report(yte, y_pred_dt, target_names=['Non-Diabetic', ' At-Risk' ,'Diabetic']))
```

```
Decision Tree Accuracy: 0.9813817673086934

Classification Report:
               precision    recall  f1-score   support

Non-Diabetic       0.99      0.99      0.99    104052
     At-Risk       0.68      0.78      0.72      3301
    Diabetic       0.99      0.94      0.96     15591

    accuracy                           0.98    122944
   macro avg       0.88      0.90      0.89    122944
weighted avg       0.98      0.98      0.98    122944
```

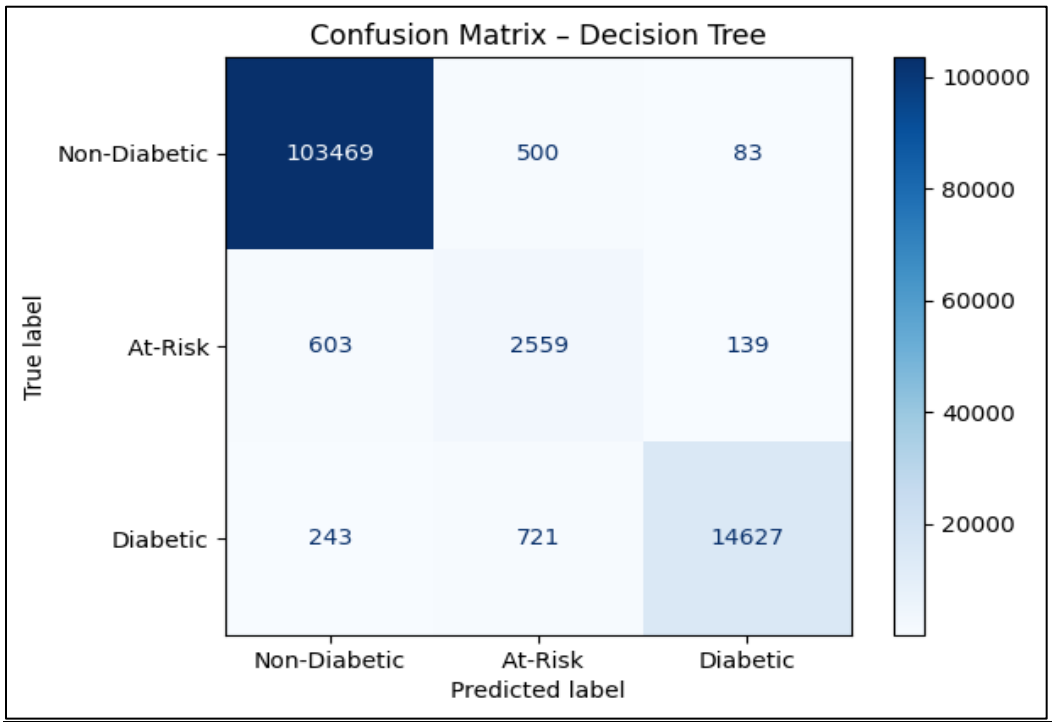
The Decision Tree model achieved an accuracy of 98.13%. The summary of the results:

Non-Diabetic: High performance with 99% precision, recall, and F1-score.

At-Risk: Moderate performance with 68% precision, but 87% recall and 76% F1-score.

Diabetic: High performance with 99% precision and 94% recall, resulting in a 96% F1-score.

Overall, the model performed well, especially for the Non-Diabetic and Diabetic classes, but the At-Risk class showed lower precision.



The Confusion Matrix shows how the Decision Tree model classified the diabetes categories:

- **Non-Diabetic:** The model correctly predicted 103,469 instances, but misclassified 500 as Non-Diabetic and 83 as At-Risk or Diabetic.
- **At-Risk:** 2,559 instances were correctly identified as At-Risk, but 603 were incorrectly classified as Non-Diabetic, and 139 as Diabetic.
- **Diabetic:** The model correctly predicted 14,627 as Diabetic, with 243 misclassified as Diabetic and 721 as At-Risk.

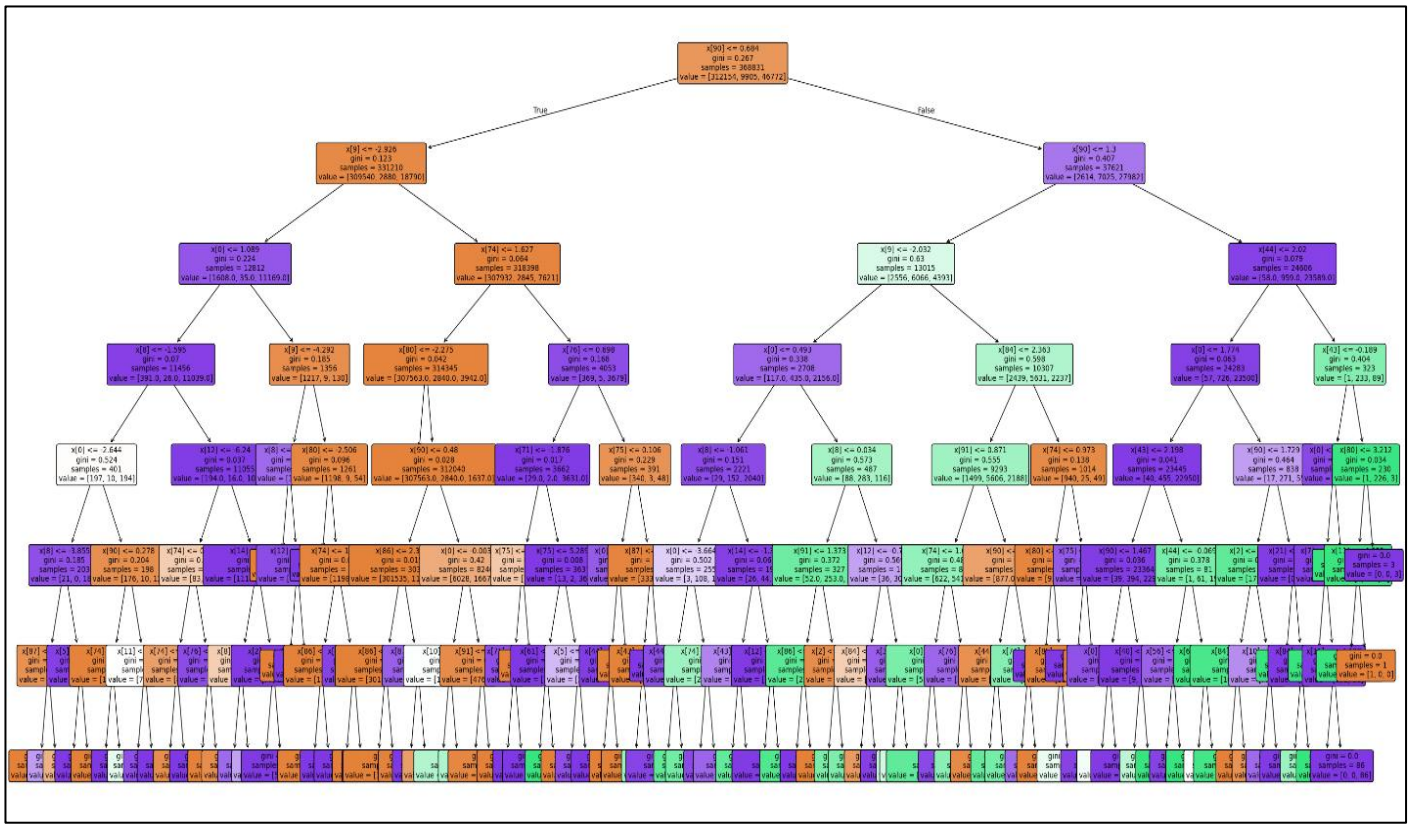
Overall, the model performed well in classifying Non-Diabetic and Diabetic categories but had some misclassifications in the At-Risk category.

KINGDOM OF SAUDI ARABIA
Ministry of Higher Education
Taibah University
College of Computer Science and Engineering (Girls Section)

جامعة طيبة
TAIBAH UNIVERSITY

المملكة العربية السعودية
وزارة التعليم العالي
جامعة طيبة
كلية علوم وهندسة الحاسب الآلي

The Visualizetion Of The Decision Tree:



For k-means model :

K-Means clustering was employed primarily for its effectiveness in analyzing feature relationships within clusters, particularly given the absence of explicit ground-truth labels. This characteristic justifies its selection as the unsupervised learning model for this study. We applied K-Means clustering following dimensionality reduction.

Initially, Principal Component Analysis (PCA) was attempted with a 95% variance retention; however, this linear approach resulted in suboptimal performance due to the complexity of the data.

Consequently, we utilized **UMAP** to project the data into a two-dimensional space. This transition led to a significant improvement in the model's quality, achieving a performance score of **0.51.**

```
score = silhouette_score(umapModel, kmeansModel.labels_)
score= round(score,2)
score
```

```
np.float32(0.52)
```

Accuracy Metrics (for classification)

- Accuracy

Measures how many predictions are correct out of all predictions.

$$Accuracy = \frac{Correct\ Predictions}{Total\ Samples}$$

- Precision

How many of the predicted positives were actually correct.

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive}$$

- Recall

How many of the actual positives we correctly identified.

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative}$$

- F1-Score

A balanced score between Precision and Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Davies–Bouldin Index (DBI)

The Davies–Bouldin Index is a simple measure that tells how good the clustering is.

$$DBI = \frac{1}{k}\sum_{i=1}^{k} \max_{j \neq i} \left(\frac{S_i + S_j}{M_{i,j}} \right)$$

It looks at two things:

- How tight each cluster is
- How far the clusters are from each other

Simple Meaning

- S_i: How spread out the points are inside cluster i.
- M_{i,j}: How far apart the centers of clusters i and j are.

The score compares how wide two clusters are to how far apart they are.

- Lower DBI = better clusters (tight and far apart)
- Higher DBI = worse clusters (spread out or overlapping)

Silhouette Score

Formula

$$Silhouette = \frac{b - a}{\max(a, b)}$$

Explanation

- a = average distance from the point to other points in the same cluster
- b = average distance from the point to points in the nearest other cluster

Interpretation

- Close to +1 → excellent clustering
- Around 0 → clusters overlap
- Negative → wrong clustering

6. Extra

K-Nearest Neighbors (KNN) – Classification Model

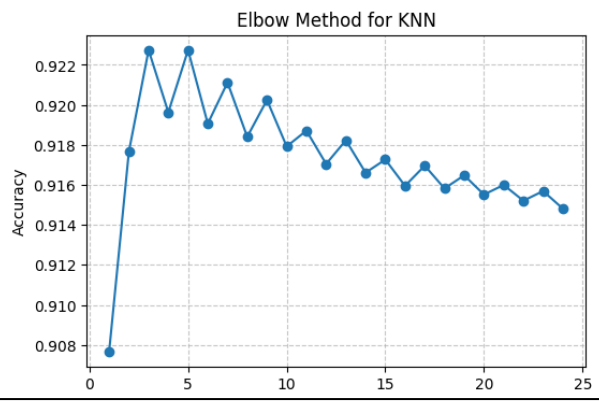
KNN is a simple classification model that predicts the class of a sample based on its closest neighbors.
To apply this model, we first selected the best value of **K** using the Elbow Method by testing different K values and observing which one gives the highest accuracy.

After finding the best K (around 5–7), we built a **KNN** pipeline with using the PCA of the goal (diabetes), then making the Elbow plot to find K that give best accuracy, and then trained the model using **X_train** and **y_train.**
Next, we predicted the test labels and evaluated the model using accuracy, classification report, and confusion matrix.

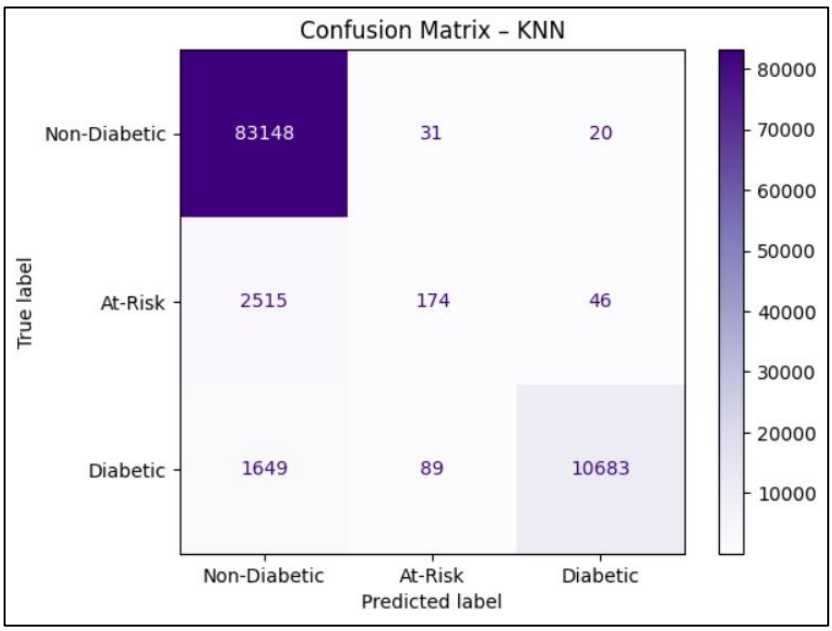
Overall, the KNN model performed well in distinguishing between Diabetic and Non-Diabetic classes, especially with the optimized K value.

Elbow graph

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

accuracies = []
k_values = range(1, 25)

for k in k_values:
    knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    accuracies.append(accuracy_score(y_test, y_pred))

plt.figure(figsize=(6,4))
plt.plot(k_values, accuracies, marker='o')
plt.title("Elbow Method for KNN")
plt.xlabel("K")
plt.ylabel("Accuracy")
plt.grid(True, linestyle="--", alpha=0.7)
plt.show()
```



```
KNN Accuracy: 0.956

Classification Report (k-NN):

              precision    recall  f1-score   support

           0       0.95      1.00      0.98     83199
           1       0.59      0.06      0.11      2735
           2       0.99      0.86      0.92     12421

    accuracy                           0.96     98355
   macro avg       0.85      0.64      0.67     98355
weighted avg       0.95      0.96      0.94     98355
```



Gradient Boosting Regressor(sleptim1)

Gradient Boosting is an ensemble model that builds multiple small trees, where each tree tries to correct the errors of the previous one. This makes it effective for handling complex and non-linear patterns in the data.

To apply this model, we selected **sleptim1** as the target and used the PCA-transformed features as input. We split the data into training and testing sets, then trained the

GradientBoostingRegressor with parameters such as learning_rate, n_estimators, and max_depth.

After training, we predicted the test values and evaluated the model using **R^2 score** and **RMSE**.

```
ybost = df_reduced['sleptim1']

X_train, X_test, y_train, y_test = train_test_split(
    X_pca, ybost, test_size=0.25,random_state=42)


train_X, test_X, train_y, test_y = train_test_split(X_pca, y,
                                                    test_size = 0.25,
                                                    random_state = 42)

gbr = GradientBoostingRegressor(loss='squared_error',
                                learning_rate=0.1,
                                n_estimators=300,
                                max_depth = 1,
                                random_state = 42,
                                max_features = 5)

gbr.fit(train_X, train_y)
pred_y = gbr.predict(test_X)
r2bost = r2_score(test_y, pred_y)

print("R² Score:", r2bost)
✓ 2m 12.9s
R² Score: 0.04213856116893122
```

However, even with a strong model like Gradient Boosting, the prediction accuracy for **sleptim1** remained very low. This shows that sleep duration has no clear pattern in the dataset, and even non-linear models could not improve the results.

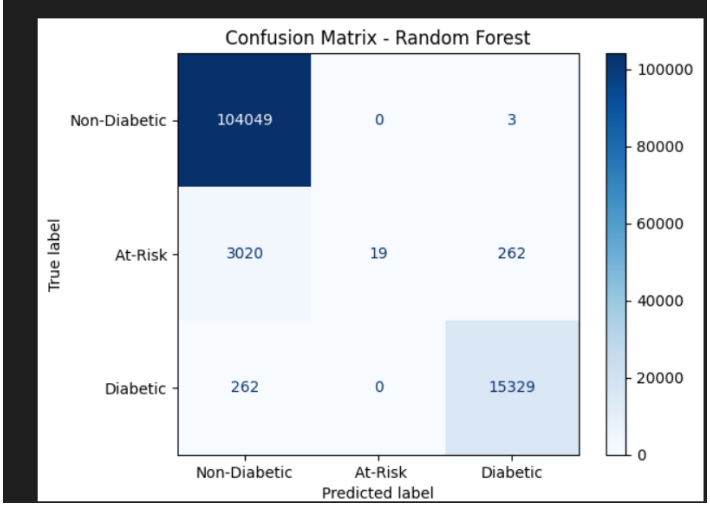
Random Forest

Random forest [4] is ensemble model that builds many separated decision trees and combines thier votes to make predictions

to evaluate more robust ensemble classifier, we implemented random forest using the same prediction target used in decision tree previously, this allowed for a comparison between different models and different approaches

```
#Random forest
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=150,
    max_depth=7,
    random_state=42
)
```

```
Classification Report:
              precision    recall  f1-score   support

Non-Diabetic       0.97      1.00      0.98    104052
     At-Risk       1.00      0.01      0.01      3301
    Diabetic       0.98      0.98      0.98     15591

    accuracy                           0.97    122944
   macro avg       0.98      0.66      0.66    122944
weighted avg       0.97      0.97      0.96    122944
```



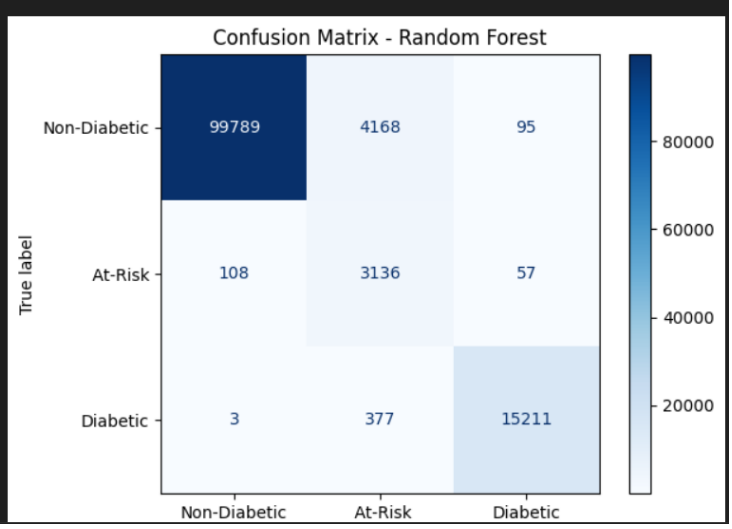
at first, the results were very poor especially with the classes ‘At-Risk’ and ‘Diabetic’ it’s struggling to identify them. it classified most of the samples as “non-diabetic”, which has

more than 100,000 samples, and failed to identify At-Risk (~3,000 samples) and Diabetic (~15,000 samples)

this means the model didn’t learn the pattern. Random forest [5]takes subset of the data for each tree, when a class has very few samples, it appear much less frequently in these subsets, so the model doesn’t learn it [5]

we applied class_weight = ‘balanced’ this parameter aims to increasing the weight of smaller classes and improving the prediction [6]

```
test accuracy:  0.9608927641853201
Classification Report:
               precision    recall  f1-score   support

Non-Diabetic       1.00      0.96      0.98    104052
     At-Risk       0.41      0.95      0.57      3301
    Diabetic       0.99      0.98      0.98     15591

    accuracy                           0.96    122944
   macro avg       0.80      0.96      0.84    122944
weighted avg       0.98      0.96      0.97    122944
```



the model improves and recognises the patterns better. for example, recall for ‘At-Risk’ jump from 0.0 to 0.95

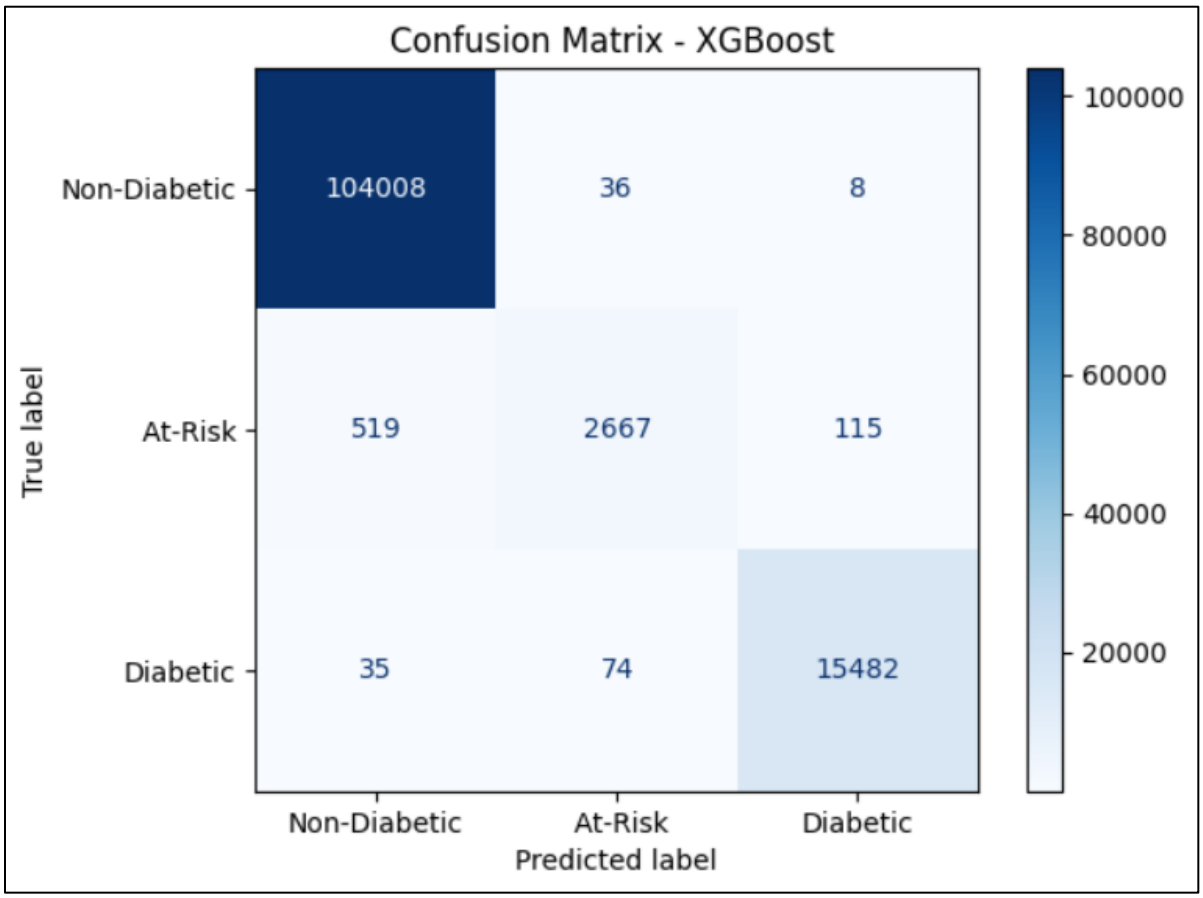
XGBoost

xgboost is boosted method that builds trees sequentially, each tree tries to fix the mistakes of previous one. this is makes XGBoost powerful especially with complex data

we also used the same target as the previous classification models

```
#XGBoost
import xgboost as xgb
from xgboost import XGBClassifier

xgb = XGBClassifier(
    n_estimators=100, #number of trees
    max_depth= 5,
    learning_rate=0.05, #determines how small each step is
    subsample=0.7, #randomly select 70% of samples in each tree
    colsample_bytree=0.7, #randomly select 70% of features in each tree
    random_state=42
    )
```

```
Train Accuracy: 0.994149081828805
Test Accuracy: 0.9935987116085372
Classification Report:
               precision    recall  f1-score   support

Non-Diabetic       0.99      1.00      1.00    104052
     At-Risk       0.96      0.81      0.88      3301
    Diabetic       0.99      0.99      0.99     15591

    accuracy                           0.99    122944
   macro avg       0.98      0.93      0.96    122944
weighted avg       0.99      0.99      0.99    122944
```



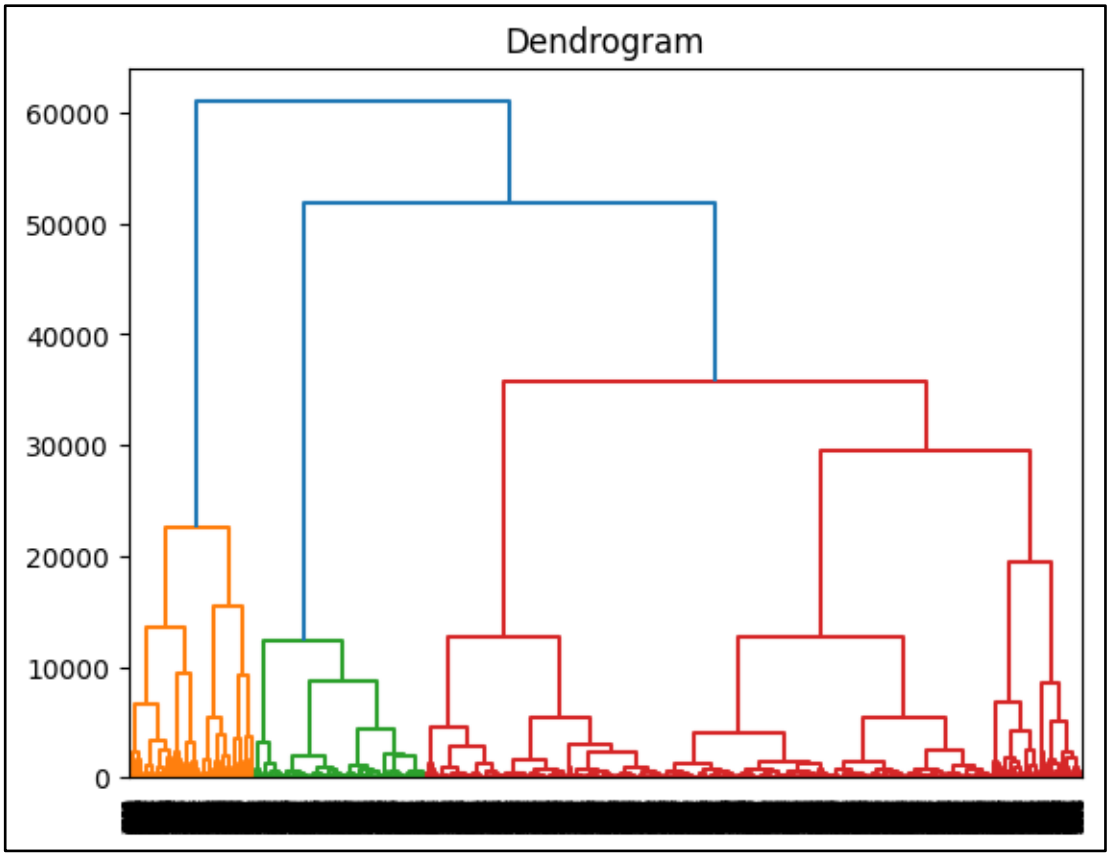
the model performed extremely well, with accuracy around 99.4%, which means it handled imbalance data much better

Comparision of Classification Models

	Decision Tree	Random Forest	XGBoost	K-nn Classification
Performance	0.98	0.96	0.99	0.96
Where it performs best	For Non-Diabetic class (precision and recall 0.99)	For Non-Diabetic and Diabetic classes	For Non-Diabetic and Diabetic classes	For Non-Diabetic and Diabetic classes
Challenge	At-Risk class (precision 0.68, recall 0.78) due to imbalance	At-Risk class (precision 0.41, recall 0.57) due to imbalance	At-Risk recall (0.83) still good but weaker than other ones	At-Risk recall (0.06) so weak and poor
Training Time	1 min, 9 sec (moderate)	9 min, 38 sec (very slow)	22 sec (very fast)	Around 1hour
Limitations	Poor handling imbalanced data	High computational cost, lowest accurcy	More complex to tune the parameters	High computational cost
Strengths	Simple structure, easy to interpretable	Reduce overfitting	Highest accuracy, fastest training	Simple to use and easy to interpret
Key parameters	max_depth = 7	n_estimators = 150, max_depth = 7, class_weight = 'balanced'	n_estimators = 100, max_depth = 5, learning_rate = 0.05	N_neighbor

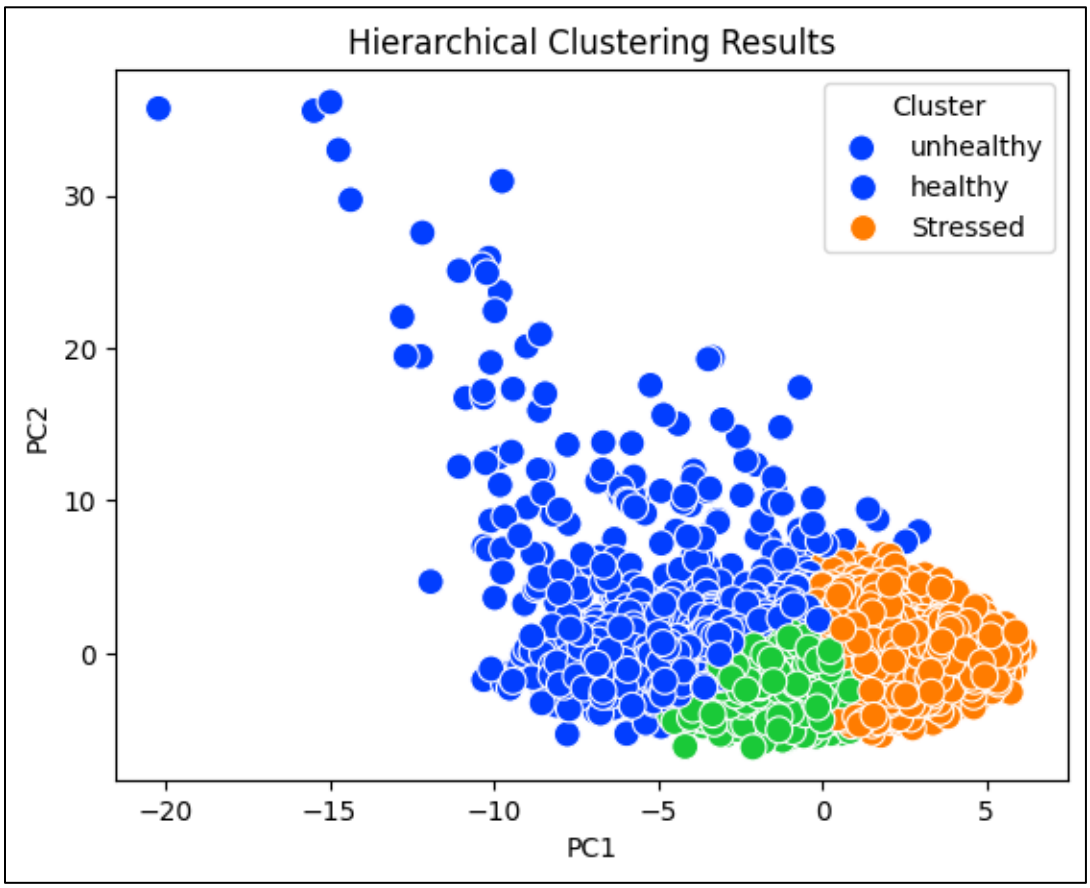
Hierarchical Model

We used a model other than K-means, which resulted in 3 clusters instead of 2. While both models yielded an identical Silhouette score, the Hierarchical [6] model offered a more precise clusters



The clusters are:

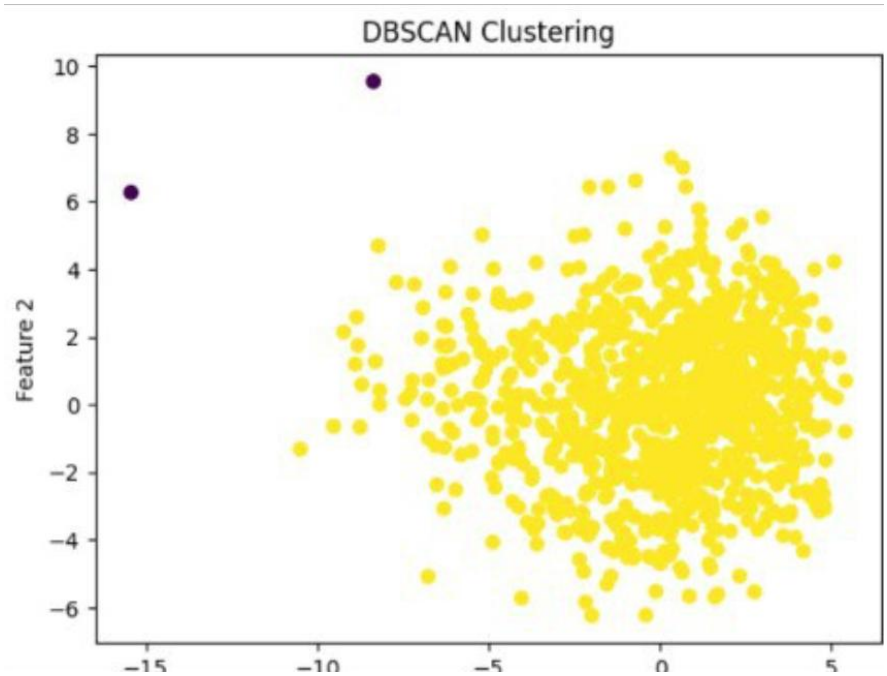
healthy, unhealthy, and healthy individuals with anxiety and sleep problems.
Therefore, Hierarchical clustering is preferred for its ability to provide a more detailed description of the relationships.



Clustering with DBSCAN:

The DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm was applied to classify the dataset. Initially, both healthy and unhealthy data points were grouped together into one cluster. After several attempts and tweaking the parameters, the algorithm identified outliers as “noise” and labeled them with a value of -1.

1. Initial Classification: The algorithm initially categorized both healthy and unhealthy points into a single cluster, meaning the points were close enough in the feature space to be considered part of the same group.
2. Outlier Detection: DBSCAN’s density-based approach identifies outliers that do not belong to any cluster. These outliers were marked as “noise” with a label of -1. This indicates that these data points were not sufficiently close to other points to form a meaningful cluster.
3. Cluster Grouping: The remaining points, which are considered either healthy or unhealthy, were grouped together into a single cluster, while the noise points remained separate. This means that, based on the given parameters (e.g., eps and min_samples), the algorithm couldn’t differentiate the two groups (healthy vs. unhealthy) but successfully identified noise.



7. Discussion and Conclusion

In this project, we applied a comprehensive machine learning pipeline to analyze the Behavioral Risk Factor Surveillance System (BRFSS) dataset, with the goal of understanding lifestyle-related health patterns and predicting key behavioral outcomes. The project demonstrated how both supervised and unsupervised learning techniques can be effectively integrated to extract meaningful insights from large-scale, noisy, and high-dimensional health data.

Through an extensive preprocessing process—including data cleaning, encoding, handling missing values, feature selection, and dimensionality reduction using PCA—we were able to transform a complex dataset into a structured form suitable for modeling. This preprocessing step proved essential for improving accuracy, stability, and interpretability across all models.

In supervised learning, Linear Regression models showed strong performance for some targets and weak performance for others, revealing that certain lifestyle factors (such as sleep duration time) lack clear predictive relationships within the dataset. The Decision Tree classifier achieved high accuracy, especially for the Non-Diabetic and Diabetic classes, although the At-Risk class remained challenging due to class imbalance and noisy patterns. Additional supervised models such as KNN, Random Forest, and XGBoost were tested to strengthen the evaluation. XGBoost in particular demonstrated exceptional accuracy, showing its advantage in handling complex and imbalanced data.

In the unsupervised part, K-Means clustering provided a general segmentation of individuals into healthy and unhealthy groups after PCA. However, Hierarchical Clustering offered more refined and interpretable clusters, distinguishing three behavior-based health profiles, including a group characterized by anxiety and sleep issues. This showed that some unsupervised models capture subtle relationships better than others.

Overall, the results highlight several key findings:
• PCA significantly enhanced model performance by reducing noise and redundancy.

• Lifestyle prediction varies greatly depending on the target variable and the underlying strength of its correlations.

• Tree-based and ensemble methods are more robust for classification within large and imbalanced health datasets.
• Unsupervised models revealed meaningful behavioral clusters, offering valuable insights beyond labeled data.

In conclusion, the project successfully demonstrated the effectiveness of combining data preprocessing with multiple machine learning approaches to analyze complex health behaviors. The insights gained can support public health research by identifying risk factors, grouping individuals based on lifestyle patterns, and improving the understanding of long-term health behaviors. Future work may include deep learning models, advanced feature engineering, or the integration of time-series health data to further enhance prediction and clustering accuracy.

8. References

[1] N. N. Phung, "Behavioral Risk Factor Surveillance System-2013," *Behavioral Risk Factor Surveillance System-2013,* 2021.

[2] "K- Fold Cross Validation in Machine Learning," 12 9 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/k-fold-cross-validation-in-machine-learning/.

[3] "umap-learn 0.5.9.post2," [Online]. Available: https://pypi.org/project/umap-learn/.

[4] "Bagging and Random Forest for Imbalanced Classification," 25 7 2023. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/bagging-and-random-forest-for-imbalanced-classification/.

[5] "RandomForestClassifier," [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html.

[6] D. Dey, "geeksforgeeks," geeksforgeeks, 8 11 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/hierarchical-clustering/.

[7] *Using KMeans and PCA do Patient Segmentation on Cancer Data and visualize with Matplotlib and more.* [Film]. Youtube, 2023.